8 Práce se soubory

V této kapitole se dozvíte:

- Jak pracovat s textovými soubory v PHP.
- Jak pracovat s adresáři v PHP.

Po jejím prostudování byste měli být schopni:

- Umět otevřít a zavřít soubor.
- Načíst data ze souboru, zapsat data do souboru, pohybovat se s kurzorem v rámci souboru.
- Umět pracovat s adresáři v PHP.

Klíčová slova této kapitoly:

soubor, čtení, zápis, pohyb kurzoru, manipulace se soubory, procházení adresáře.

Doba potřebná ke studiu: 4 hodiny



Průvodce studiem

Studium této kapitole není náročné, kapitola seznamuje čtenáře s předdefinovanými funkcemi pro práci se soubory a s adresáři. V kapitole se naučíte otevřít a zavřít textový soubor, načítat obsah souboru, zapisovat do souboru, manipulovat se souborem jako s celkem a nakonec se naučíte pracovat s adresáři.

Při vytváření webových aplikací je někdy potřeba ukládat si data získaná od uživatelů. V některých případech je zbytečné použít pro uložení dat databázi. Místo databáze můžeme k uložení dat použít textové soubory. Jazyk PHP umožňuje pracovat s textovými soubory – vytvářet nebo odstraňovat soubory, měnit jejich obsah, číst ze souborů. Kromě toho je v PHP možné pracovat s adresářovou strukturou – procházet obsah složky, vytvořit složku nebo ji přejmenovat.

8.1 Otevření a zavření souboru



Základní operací při práci se soubory je otevření souboru. Abychom mohli ze souboru číst nebo do něj zapisovat je potřeba ho nejprve otevřít. Po ukončení práce se souborem je samozřejmě potřeba soubor zavřít. K otevření souboru je v PHP k dispozici funkce `fopen()`, jejíž základní hlavička je ve tvaru:

```
resource fopen(string název_souboru, string režim)
```

Prvním parametrem je název souboru, který chceme otevřít. Může se jednat o lokální souboru v rámci adresářové struktury serveru, ale také o soubor otevřený v rámci internetu pomocí protokolu HTTP nebo FTP. Druhým

parametrem určujeme, v jakém režimu se má soubor otevřít – může se jednat o čtení ze souboru, zápis do souboru nebo přidávání do souboru. K určení požadovaného režimu slouží hodnoty uvedené v následující tabulce. Kromě dvou povinných parametrů je možné ve funkci `fopen()` použít ještě dva volitelné parametry. První volitelný parametr může být nastaven na hodnotu `true` nebo `false` a určuje, zda se zadaný soubor bude hledat i v adresáři zadaných v `include_path`. Druhý volitelný parametr určuje kontext, který souvisí s používáním proudů při práci se soubory.

Hodnota	**Popis**
`r`	Otevře soubor pouze pro čtení. Data budou zapsána na začátek souboru.
`r+`	Otevře soubor pro čtení a zápis. Data budou zapsána na začátek souboru.
`w`	Otevře soubor pouze pro zápis. Stávající obsah souboru bude vymazán. Pokud soubor neexistuje, pokusí se jej PHP vytvořit.
`w+`	Otevře soubor pro čtení a zápis. Stávající obsah souboru bude vymazán. Pokud soubor neexistuje, pokusí se jej PHP vytvořit.
`a`	Otevře soubor pouze pro přidávání. Data budou přidána na konec souboru. Pokud zadaný soubor neexistuje, pokusí se jej PHP vytvořit.
`a+`	Otevře soubor pro přidávání a čtení. Data budou přidána na konec souboru. Pokud zadaný soubor neexistuje, pokusí se jej PHP vytvořit.
`x`	Vytvoří a otevře soubor pouze pro zápis. Data budou zapsána na začátek souboru. Pokud zadaný soubor existuje, volání funkce `fopen()` selže a vrátí se hodnota `false`.
`x+`	Vytvoří a otevře soubor pro čtení a zápis. Data budou zapsána na začátek souboru. Pokud zadaný soubor existuje, volání funkce `fopen()` selže a vrátí se hodnota `false`.
`c`	Otevře soubor pouze pro zápis. Pokud zadaný soubor neexistuje, bude vytvořen. Pokud soubor existuje není jeho obsah smazán, ani nedojde k chybě při volání funkce `fopen()`. Data budou zapsána na začátek souboru.
`c+`	Otevře soubor pro čtení a zápis. Pokud zadaný soubor neexistuje, bude vytvořen. Pokud soubor existuje není jeho obsah smazán, ani nedojde k chybě při volání funkce `fopen()`. Data budou zapsána na začátek souboru.

V případě, že funkci `fopen()` použijeme k otevření jiného souboru v rámci Internetu (pomocí protokolu HTTP), můžeme tento soubor otevřít pouze pro čtení. K hodnotám určujícím režim otevření souboru můžeme přidat příznak `b`, pomocí kterého označíme otevíraný soubor jako binární (například se může jednat o obrázek).

Pokud otevření zadaného souboru proběhne v pořádku, vrátí funkce `fopen()` identifikátor souboru (ukazatel na soubor), pomocí něho provádíme další práci se souborem. V případě, že pokus o otevření souboru nebyl úspěšný, vrátí funkce `fopen()` hodnotu `0` (`false`).

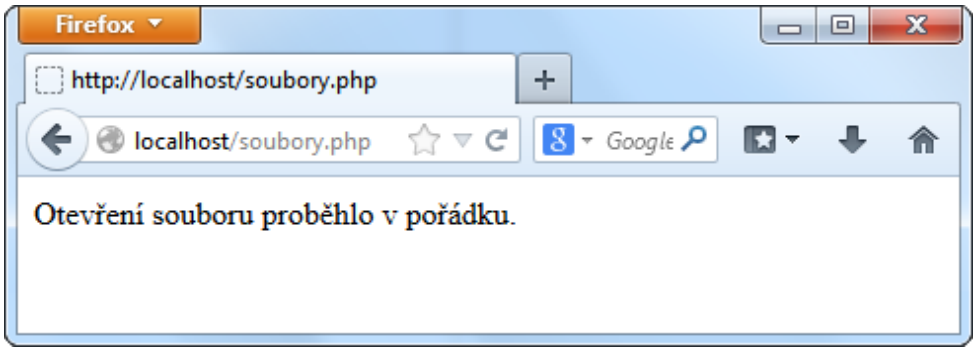
Následující výpis obsahuje ukázku použití funkce `fopen()`. Pokud soubor s názvem `aaa.txt` neexistuje, bude vytvořen a otevřen pro čtení i zápis.

Výpis 8.1 Otevření souboru.



```
<?php
  if ($soubor = fopen("aaa.txt", "w+")) {
    // soubor otevren, muzeme provest cteni nebo
     zapis
    echo "Otevření souboru proběhlo v pořádku.";
  } else {
    // chyba
    echo "Chyba při otevírání souboru.";
  }
?>
```

Název souboru je uveden relativně, takže adresářem pro hledání souboru bude adresář, ve kterém se nachází soubor `soubory.php` (v tomto případě je to kořenový adresář webového serveru). Pokud soubor neexistuje a má být vytvořen, je potřeba, aby byla kořenovému adresáři serveru nastavena přístupová práva pro vytváření souborů. Na obrázku, který následuju je zobrazeno okno prohlížeče pro provedení příkazů uvedených ve výpisu 8.1.



Obrázek 8.1 Otevření souboru.

Po ukončení práce se souborem je potřeba soubor zavřít. K zavření souboru slouží funkce `fclose()`, jejíž hlavička vypadá následovně:



```
bool fclose(resource soubor)
```

Parametrem funkce je ukazatel na soubor, který chceme zavřít. Pokud uzavření souboru proběhne úspěšně, vrátí funkce `true`, jinak vrátí hodnotu `false`.

Následující výpis ukazuje použití funkce `fclose()`, předpokládá se, že v proměnné `soubor` je uložen platný odkaz na existující soubor.

Výpis 8.2 Zavření souboru.



```
<?php
  if (fclose($soubor)) {
    echo "Zavření souboru proběhlo v pořádku.";
  } else {
    echo "Chyba při zavírání souboru.";
  }
?>
```

V některých případech může být užitečné nejprve otestovat, zda požadovaný soubor existuje. K testování existence souboru slouží funkce `file_exists()`, jejímž parametrem je název souboru. Pokud soubor existuje, funkce vrátí `true`, jinak vrátí `false`.

8.2 Práce s obsahem souboru

Po otevření souboru ke čtení je kurzor v souboru nastaven na začátek souboru. Ke zjištění, zda je kurzor na konci souboru slouží funkce `feof()`, která má jeden parametr – platný ukazatel na otevřený soubor. Funkce vrací `true`, pokud je kurzor na konci souboru, jinak vrací `false`. Ke zjištění velikosti souboru slouží funkce `filesize()`, jejímž parametrem je název souboru. Funkce vrací velikost souboru v bytech.

8.2.1 Čtení ze souboru



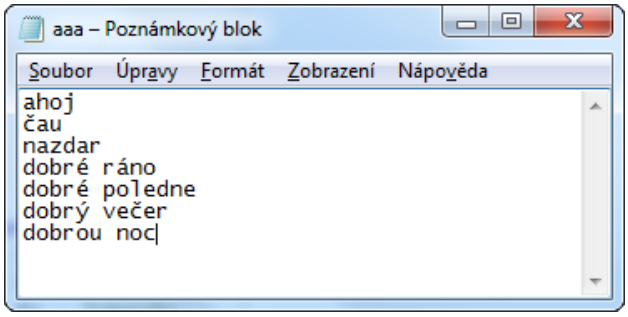
Čtení ze souborů můžeme provádět několika způsoby. Obsah souboru můžeme načíst celý najednou a dále s ním pracovat a nebo můžeme obsah souboru číst postupně – můžeme načítat řetězce o zadané délce, můžeme načíst jeden znak nebo jeden řádek.

K načtení a vytištění celého obsahu souboru slouží funkce `fpassthru()`. Parametrem funkce je platný ukazatel na otevřený soubor. Funkce načte obsah

souboru a odešle jej na výstupní proud (např. do okna prohlížeče). Pokud je čtení úspěšné, vrátí funkce počet načtených znaků. Pokud dojde při čtení souboru k chybě, funkce vrátí hodnotu `false`.

Druhou možností načtení celého obsahu souboru najednou je použití funkce `file()`, která má jako parametr název souboru. Funkce `file()` načte celý obsah souboru a uloží jej do pole. Prvky pole tvoří jednotlivé řádky souboru, počet prvků pole je tedy roven počtu řádků souboru. Kromě názvu souboru má funkce ještě dva nepovinné parametry, kterými se ale nebudeme v tomto textu zabývat.

Ve zbývající části kapitoly budeme pracovat se souborem `aaa.txt`. Soubor otevřený v aplikaci Poznámkový blok je zobrazený na následujícím obrázku.



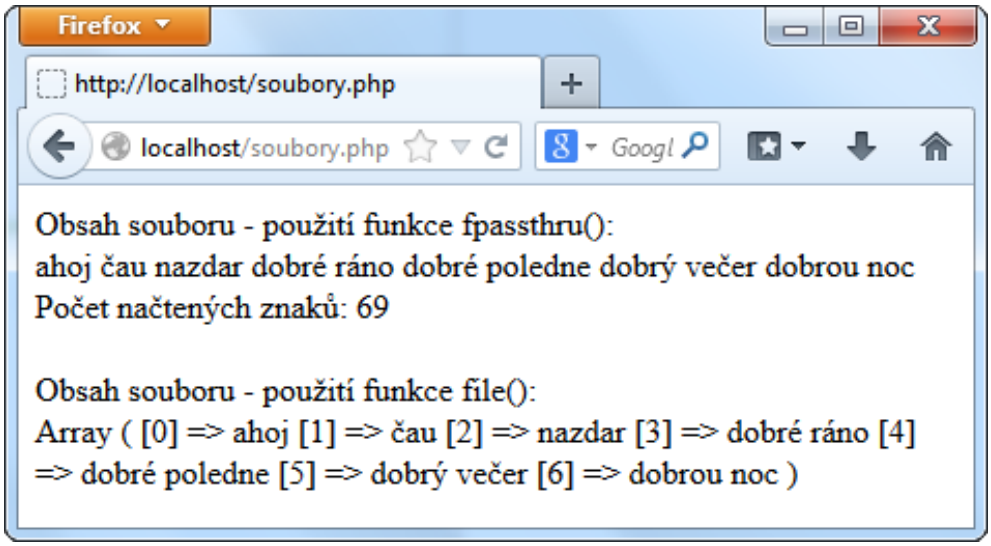
Obrázek 8.2 Obsah souboru aaa.txt.

Následující výpis ukazuje použití funkcí `fpassthru()` a `file()`. Obrázek pod výpisem pak zobrazuje výsledek zobrazený v prohlížeči.

Výpis 8.3 Použití funkcí fpassthru() a file().



```
<?php
  if ($soubor = fopen("aaa.txt", "r")) {
    echo "Obsah souboru - použití funkce
     fpassthru():<br>";
    $znaky = fpassthru($soubor);
    echo "<br>Počet načtených znaků: $znaky<br><br>";
    fclose($soubor);
  } else {
    echo "Chyba při otevírání souboru.";
  }
  $pole = file("aaa.txt");
  echo "Obsah souboru - použití funkce file():<br>";
  print_r($pole);
?>
```



Obrázek 8.3 Použití funkcí fpassthru() a file().

Při zpracování obsahu souboru po částech můžeme načíst jeden znak nebo zadaný počet znaků. Pro načtení jednoho znaku ze souboru slouží funkce `fgetc()`, jejímž parametrem je platný ukazatel na otevřený soubor. Funkce vrací řetězec o délce jeden znak.

Jako alternativu můžeme použít funkci `fgets()`, která ze souboru načte jeden řádek textu. Funkce má jeden povinný a jeden volitelný parametr. Povinným parametrem je platný ukazatel na otevřený soubor, volitelným parametrem je celé číslo určující počet načtených znaků. Hlavička funkce je ve tvaru

```
string fgets(resource soubor, int délka)
```

Pokud nebyl zadán parametr `délka`, načte se text do konce aktuálního řádku (případně do konce souboru). Pokud byl zadán parametr `délka`, je ze souboru načteno (`délka - 1`) znaků. V případě, že je před načtením určeného počtu znaků dosaženo konce řádku nebo konce souboru, vrátí funkce menší počet znaků. Další velmi podobnou funkcí je funkce `fgetss()`, která má stejné parametry jako funkce `fgets()`. Rozdíl mezi těmito funkcemi je v tom, že funkce `fgetss()` odstraní z načteného řetězce HTML značky. Pokud zadáme parametr `délka`, jsou tyto značky započítány do délky načteného řetězce.

Poslední funkcí pro čtení obsahu souboru, kterou si zde uvedeme, je funkce `fread()`. Tato funkce má stejné parametry jako funkce `fgets()`, tj. ukazatel na otevřený soubor a počet znaků, které se mají načíst. Na rozdíl od funkce `fgets()` jsou oba parametry funkce `fread()` povinné. Dalším rozdílem je, že ze souboru bude načten počet znaků, který je roven parametru

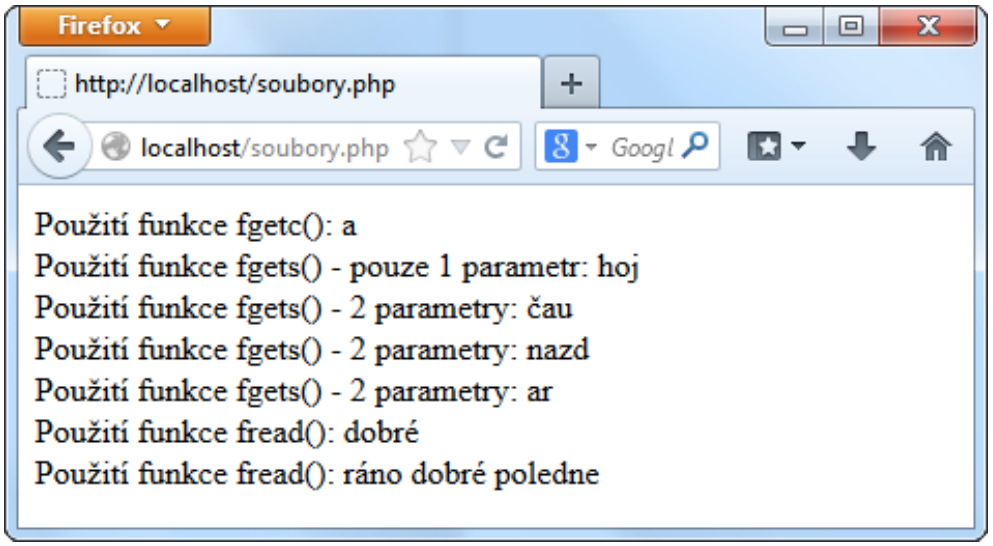
délka a že čtení znaků neskončí při dosažení konce řádku. Samozřejmě ale skončí po dosažení konce souboru.

Následující výpis zobrazuje ukázku použití uvedených funkcí, obrázek pod výpisem pak výsledek zobrazený v okně prohlížeče.

Výpis 8.4 Použití funkcí fgetc(), fgets() a fread().



```
<?php
  if ($soubor = fopen("aaa.txt", "r")) {
    echo "Použití funkce fgetc(): " . fgetc($soubor)
     . "<br>";
    echo "Použití funkce fgets() - pouze 1 parametr:
     " . fgets($soubor) . "<br>";
    echo "Použití funkce fgets() - 2 parametry: " .
     fgets($soubor, 10) . "<br>";
    echo "Použití funkce fgets() - 2 parametry: " .
     fgets($soubor, 5) . "<br>";
    echo "Použití funkce fgets() - 2 parametry: " .
     fgets($soubor, 5) . "<br>";
    echo "Použití funkce fread(): " . fread($soubor,
     5) . "<br>";
    echo "Použití funkce fread(): " . fread($soubor,
     20) . "<br>";
    fclose($soubor);
  } else {
    echo "Chyba při otevírání souboru.";
  }
?>
```



Obrázek 8.4 Použití funkcí fgetc(), fgets() a fread().

8.2.2 Zápis do souborů



Pro zápis textu do souboru slouží funkce `fwrite()`, jejíž hlavička má následující tvar:

```
int fwrite(resource soubor, string text, int délka)
```

Prvním parametrem funkce je platný ukazatel na otevřený soubor. Druhým parametrem je řetězec, který chceme do souboru zapsat. Poslední parametr `délka` je volitelný a udává počet znaků, které se mají zapsat do souboru. Pokud není tento parametr zadán, zapíše se celý řetězec uvedený v parametru `text`. V případě, že zápis do souboru proběhne úspěšně, vrátí funkce `fwrite()` počet zapsaných bytů. V případě neúspěchu vrátí `false`.

Při zápisu do souboru je potřeba mít brát zřetel na to, v jakém režimu byl otevřen cílový soubor. Na použitém režimu závisí to, kam se data do souboru zapíší.

Druhou funkcí, kterou můžeme použít pro zápis do souboru, je funkce `fputs()`. Tato funkce je ale pouze aliasem funkce `fwrite()`, tj. má stejné parametry i návratovou hodnotu a liší se pouze v názvu.

Následující výpis zobrazuje ukázku použití funkce `fwrite()`.

Výpis 8.5 Ukázka použití funkcí fwrite() a fputs().



```
<?php
  if ($soubor = fopen("bbb.txt", "w")) {
    $pocet = fwrite($soubor, "První text");
    echo "Počet zapsaných znaků: $pocet<br>";
    $pocet = fputs($soubor, "Druhý text");
    echo "Počet zapsaných znaků: $pocet<br>";
    $pocet = fwrite($soubor, "Třetí text", 5);
    echo "Počet zapsaných znaků: $pocet<br>";
    fclose($soubor);
  } else {
    echo "Chyba při otevírání souboru.";
  }
?>
```

8.2.3 Pohyb uvnitř souborů



Při práci s obsahem souboru je někdy potřeba přesunout pozici kurzoru v souboru na jiné místo, na kterém má být provedeno čtení nebo zápis. Pro přesun na začátek souboru je určena funkce `rewind()`, jejímž jediným

parametrem je platný ukazatel na otevřený soubor. Pokud přesun na začátek souboru proběhne úspěšně, funkce vrátí hodnotu `true`, jinak vrátí `false`. V případě, že soubor byl otevřen v režimu přidávání (tj. režim byl „`a`“ nebo „`a+`“), při zápisu se vždy data zapíší na konec souboru, a to bez ohledu na to, kam jsme přesunuli kurzor v rámci souboru.

Pro zjištění aktuální pozice kurzoru v souboru je určena funkce `ftell()`, která má jako parametr platný ukazatel na otevřený soubor. Pokud při zjišťování pozice kurzoru nastane chyba, vrátí funkce hodnotu `false`. V opačném případě vrátí celé číslo udávající pozici kurzoru v rámci souboru.

Funkce `fseek()` slouží pro umístění kurzoru v rámci souboru na konkrétní pozici. Pokud umístění kurzoru proběhne úspěšně, vrací funkce hodnotu `0`, jinak vrací hodnotu `-1`. Funkce `fseek()` má tři parametry. Prvním parametrem je platný ukazatel na otevřený soubor, druhým parametrem je pozice (počet znaků od začátku souboru) a třetím parametrem je nastavení, jak se bude počítat výsledná pozice. První dva parametry jsou povinné, třetí parametr je nepovinný.

Poslední funkcí, která souvisí s pohybem kurzoru v rámci souboru, je funkce `foef()`, která již byla zmíněna v úvodu této kapitoly. Funkce zjišťuje, zda se kurzor nachází na konci souboru.

8.3 Manipulace se soubory



Při práci se soubory nemusíme pracovat pouze s obsahem souboru, ale můžeme pracovat i se souborem jako s celkem. Jazyk PHP umožňuje vytvořit kopii souboru, přejmenovat soubor a odstranit soubor.

Pro zkopírování souboru je určena funkce `copy()`, která má dva parametry. Prvním parametrem je název zdrojového souboru, resp. cesta k tomuto souboru. Druhým parametrem je název cílového souboru. Pokud vytvoření kopie souboru proběhne úspěšně, funkce vrátí hodnotu `true`. V opačném případě vrátí hodnotu `false`.

Přejmenování souborů je možné provést pomocí funkce `rename()`. Parametry této funkce jsou starý název souboru a nový název souboru. Pokud jako nový název zadáme název již existujícího souboru, bude tento soubor přepsán. Stejně jako předchozí funkce, i funkce `rename()` vrací v případě úspěchu hodnotu `true`, jinak vrací `false`.

Pokud chceme soubor odstranit, použijeme k tomuto účelu funkci `unlink()`. Parametrem funkce je název souboru, který chceme odstranit. Pokud odstranění souboru proběhne úspěšně, funkce vrací `true`, jinak vrací `false`.

Kromě manipulace se soubory je někdy potřeba zjišťovat vlastnosti souborů. K otestování, zda zadaný soubor nebo adresář vůbec existuje slouží funkce `file_exists()`. Ke zjištění velikosti souboru slouží funkce `filesize()`. Obě funkce již byly zmíněny v této kapitole. Další vlastností souboru, kterou můžeme zjistit, je typ souboru. K určení typu souboru je určena funkce `filetype()`, která má jako parametr název souboru a vrací textový řetězec označující typ souboru. V případě úspěchu vrací funkce jednu z osmi předdefinovaných hodnot, v případě neúspěchu vrací `false`. Návratové hodnoty označují o jaký typ souboru se jedná. Hodnota `file` určuje, že se jedná o normální soubor, hodnota `dir` určuje, že se jedná o adresář, hodnota `unknown` určuje, že typ souboru se nepodařilo zjistit. Zbylé hodnoty určují speciální typy souborů.

Ke zjištění, zda je zadaný soubor určitého typu pak slouží sada logických funkcí, které vrací `true` v případě, že zadaný soubor je určitého typu, nebo `false`, pokud je jiného typu. Všechny funkce mají jeden parametr, a tím je název souboru. Funkce `is_dir()` zjišťuje, zda se jedná o adresář. Funkce `is_file()` zjišťuje, zda se jedná o normální soubor. Funkce `is_executable()` testuje, zda se jedná o spustitelný soubor. Kromě toho je možné zjistit, zda lze zadaný soubor nebo adresář číst a nebo do něj zapisovat. K tomuto účelu slouží funkce `is_readable()` – čtení a `is_writable()` – zápis.

Výpis 8.6 Ukázka manipulace se soubory.

```
<?php
  // vytvoreni kopie souboru ve stejnem adresari
  copy("./aaa.txt", "./aaa-kopie.txt");
  // vytvoreni kopie souboru v jinem adresari –
   adresar musí existovat
  copy("./aaa.txt", "./pokus/aaa-kopie.txt");
  // prejmenovani souboru
  rename("./pokus/aaa-kopie.txt", "./pokus/aaa-
   nove.txt");
  // odstraneni souboru
  unlink("aaa-kopie.txt");
  // zjisteni typu souboru
  echo filetype("aaa.txt");
```



```
  echo is_dir("./pokus");
  echo is_file("aaa.txt");
  echo is_readable("aaa.txt");
  echo is_writable("aaa.txt");
?>
```

8.4 Práce se adresáři



Jazyk PHP umožňuje pracovat také s adresáři. Můžeme nastavit aktuální adresář, procházet obsah adresáře, vytvořit nebo odstranit adresář.

Při tvorbě aplikací v jazyce PHP je vždy nastaven aktuální pracovní adresář. ke zjištění názvu aktuálního adresáře slouží funkce `getcwd()`. Tato funkce nemá žádné parametry a vrací textový řetězec představující aktuální pracovní adresář. V případě chyby funkce vrací `false`. K nastavení aktuálního adresáře slouží funkce `chdir()`. Parametrem funkce řetězec určující cestu k adresáři. V případě úspěchu funkce vrací hodnotu `true`, jinak vrací `false`. Název nového aktuálního adresáře může být uveden absolutně (celá cesta k adresáři) nebo relativně k právě aktuálnímu adresáři.

Procházení obsahu adresáře je možné dvěma způsoby. Prvním způsobem je použití funkcí pro procházení adresáře, druhým způsobem je použití třídy `Directory`. Oba způsoby jsou velmi podobné.

Při použití funkcí pro procházení adresářů musíme nejprve otevřít požadovaný adresář. K otevření adresáře je určena funkce `opendir()`, jejímž parametrem je textový řetězec představující cestu k adresáři. Cesta může být zadána absolutně nebo relativně. Funkce vrací ukazatel na otevřený adresář. K procházení položek otevřeného adresáře slouží funkce `readdir()`, která vrací název následující položky v adresáři. Parametrem funkce je platný ukazatel na otevřený adresář. Funkce `readdir()` se pohybuje v adresáři od začátku do konce. Pokud se potřebujeme vrátit na začátek adresáře, použijeme funkci `rewinddir()`. Parametrem funkce je platný ukazatel na otevřený adresář. Funkce nevrací žádnou hodnotu (resp. vrací `void`). Po ukončení práce s adresářem je potřeba zavřít otevřený adresář, a to pomocí funkce `closedir()`. Parametrem je opět platný ukazatel na otevřený adresář a návratovým typem funkce je `void`.

Druhým způsobem procházení adresáře je použití třídy `Directory`. Funkce `dir()`, jejímž parametrem je textový řetězec představující cestu k adresáři, vrací instanci třídy `Directory`. Třída `Directory` obsahuje tři metody,

které slouží k pohybu v adresáři. Metoda `read()` má stejnou funkčnost jako funkce `readdir()`, metoda `rewind()` je ekvivalentem funkce `rewinddir()` a metoda `close()` je ekvivalentní k metodě `closedir()`. Kromě uvedených metod obsahuje třída `Directory` ještě dvě členské proměnné – proměnná `$path` obsahuje řetězec představující otevřený adresář, proměnná `$handle` obsahuje ukazatel na otevřený adresář.

Vytvoření nového adresáře je možné pomocí metody `mkdir()`. Parametrem je textový řetězec představující nový adresář. Odstranění adresáře je možné pomocí metody `rmdir()`, jejímž parametrem je opět textový řetězec představující adresář. Obě metody vrací `true`, pokud je operace úspěšná, nebo `false` v případě neúspěchu.

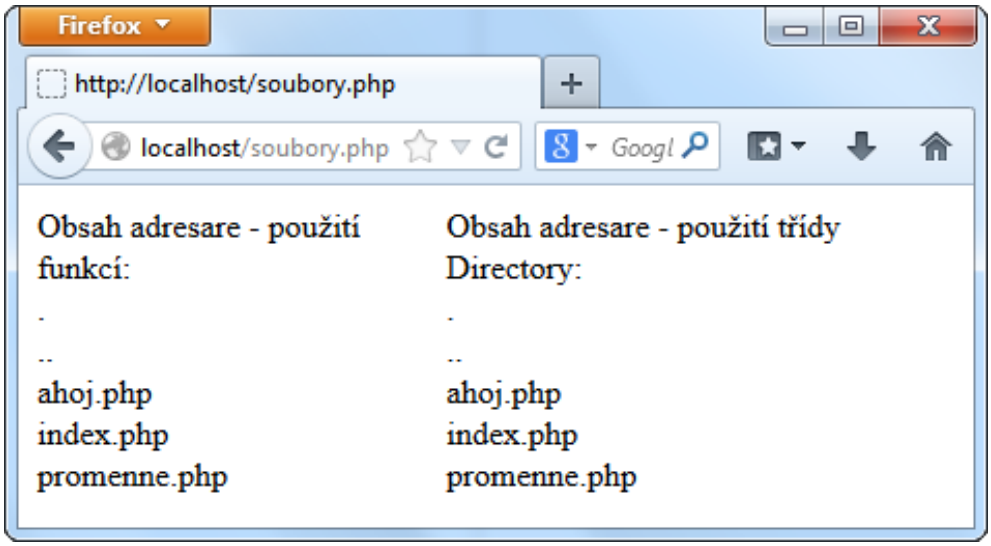
Následující výpis zobrazuje ukázku práce s adresáři, obrázek pod výpisem pak výstup v prohlížeči.

Výpis 8.7 Ukázka práce s adresáři.

```
<?php
  echo '<table border="0" cellspacing="0"><tr><td>';
  $adresar = opendir("pokus");
  echo "Obsah adresare - použití funkcí:<br>";
  while (false !== ($zaznam = readdir($adresar))) {
    echo "$zaznam<br>";
  }
  closedir($adresar);

  echo '</td><td>';
  // pouziti tridy Directory
  $adresar = dir("pokus");
  echo "Obsah adresare - použití třídy
   Directory:<br>";
  while (false !== ($zaznam = $adresar->read())) {
    echo "$zaznam<br>";
  }
  $adresar->close();
  echo '</td></tr></table>';
?>
```



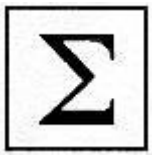


Obrázek 8.5 Výpis obsahu adresáře.



Kontrolní otázky:

1. Jaké parametry má funkce `fopen()` a co je jejím návratovým typem?
2. Jaký je rozdíl mezi funkcemi `fgetc()`, `fgets()` a `fread()`?
3. Jaký je rozdíl mezi funkcemi `fgets()` a `fgetss()`?
4. Jakými způsoby je možné vypsat obsah konkrétního adresáře?



Shrnutí obsahu kapitoly

V této kapitole jste s předdefinovanými funkcemi v PHP, které slouží pro práci se soubory. K otevření souboru slouží funkce `fopen()`, k jeho zavření pak `fclose()`. Čtení ze souboru je možné provést pomocí funkcí `fgetc()`, `fgets()`, `fgetss()` nebo `fread()`. Zápis do souboru je možný pomocí funkce `fwrite()`. Funkce `copy()`, `rename()` a `unlink()` slouží ke zkopírování, přejmenování a odstranění souboru. Výpis obsahu adresáře je možný dvěma způsoby – pomocí funkcí nebo pomocí třídy `Directory`.